

Laboratorio 6 - Esercizi avanzati sulla concorrenza

L'obiettivo di questo laboratorio è di completare l'esplorazione dei costrutti che Rust mette a disposizione per la programmazione concorrente, costruendo su quanto visto nel Laboratorio 5. Gli esercizi sono di complessità crescente: i primi introducono i nuovi strumenti (`Condvar`, canali `mpsc`, `crossbeam`) in isolamento, mentre gli ultimi richiedono di combinare più costrutti per risolvere problemi più articolati. Nella cartella `src/` troverete un file sorgente per ciascun esercizio, con al suo interno il codice di partenza (boilerplate) e una breve descrizione simile a quella sottostante. Per verificare la correttezza della vostra implementazione potete utilizzare i test forniti, eseguendo su terminale `cargo test <nome esercizio>`, ad esempio `cargo test es1_barrier`. Invece, per eseguire i test di tutti gli esercizi, potete semplicemente eseguire `cargo test` senza alcun argomento.

Esercizio 1 - Barriera di sincronizzazione

Si vuole realizzare una **barriera di sincronizzazione** riusabile, simile a `std::sync::Barrier` ma implementata a mano. Una barriera è un punto di rendezvous in cui `N` thread si attendono reciprocamente: nessuno prosegue finché tutti e `N` non hanno raggiunto la barriera. Una volta che l'ultimo thread arriva, tutti vengono sbloccati e la barriera può essere riutilizzata per un secondo turno.

Definire la struct e i metodi:

```
Rust
pub struct MyBarrier { /* campi */ }
impl MyBarrier {
    pub fn new(n: usize) -> Self;
    pub fn wait(&self);
}
```

Requisiti:

- La barriera deve essere **riusabile**: dopo che `N` thread l'hanno attraversata, deve poter essere usata di nuovo per un nuovo gruppo di `N` thread.
- Il metodo `wait()` deve bloccare il thread chiamante senza consumare cicli di CPU finché tutti gli altri `N-1` partecipanti non hanno chiamato a loro volta `wait()`.
- Si gestisca correttamente il caso in cui un thread arrivi alla barriera mentre un altro gruppo sta ancora uscendo dal giro precedente.

Domande:

- Quali strumenti di sincronizzazione conviene utilizzare per implementare la barriera? Perché?

- È più corretto svegliare un solo thread alla volta o tutti insieme quando l'ultimo thread arriva? Perché?
- Cosa sono le notifiche spurie e come ci si protegge da esse in questo esercizio?

Esercizio 2 - Canale MPSC implementato a mano

Si vuole realizzare a mano un canale di comunicazione asincrono tra thread, con semantica *multi-producer single-consumer*, analogo (in forma semplificata) a quello offerto da `std::sync::mpsc`. Il canale è composto da due estremi distinti: un `Sender<T>` e un `Receiver<T>`. I produttori inseriscono messaggi tramite `send`; il consumatore li estrae nello stesso ordine di invio tramite `recv`. Quando il consumatore prova a leggere da un canale vuoto, si blocca passivamente in attesa di un messaggio. Quando tutti i Sender sono stati distrutti, il consumatore ne deve essere informato e le sue chiamate a `recv` devono restituire un errore. Specularmente, quando il `Receiver` è stato distrutto, i `Sender` devono accorgersene e le loro chiamate a `send` devono fallire.

Definire la funzione di creazione e i tipi:

Rust

```
pub fn channel<T>() -> (Sender<T>, Receiver<T>);

pub struct Sender<T> { /* campi */ }
pub struct Receiver<T> { /* campi */ }

#[derive(Debug)]
pub struct SendError<T>(pub T);

#[derive(Debug, PartialEq, Eq)]
pub struct RecvError;

#[derive(Debug, PartialEq, Eq)]
pub enum TryRecvError { /* errori */ }

impl<T> Sender<T> {
    pub fn send(&self, item: T) -> Result<(), SendError<T>>;
}

impl<T> Receiver<T> {
    pub fn recv(&self) -> Result<T, RecvError>;
    pub fn try_recv(&self) -> Result<T, TryRecvError>;
}
```

Requisiti:

- Il canale è **unbounded**: `send` non si blocca mai per motivi di capacità.
- `Sender<T>` deve implementare `Clone`: clonarlo permette di avere più produttori.
- `recv` deve attendere senza consumare CPU.
- Quando l'ultimo `Sender` viene distrutto e la coda interna è vuota, le chiamate a `recv` devono restituire `Err(RecvError)`.
- Quando il `Receiver` viene distrutto, le successive chiamate a `send` devono restituire `Err(SendError(item))`, restituendo al chiamante l'elemento che non è stato possibile consegnare.
- L'ordine di consegna dei messaggi deve essere FIFO rispetto a ciascun `Sender`.

Domande:

- Come si fa ad accorgersi che tutti i `Sender` sono stati distrutti?
- Come si fa ad accorgersi che il `Receiver` è stato distrutto? In quel caso, dove conviene segnalare la cosa ai `Sender`?
- Cosa cambierebbe se si volesse rendere il canale *bounded*, ovvero con una capacità massima oltre la quale `send` deve bloccarsi?

Esercizio 3 - Pipeline a stadi con canali `mpsc`

Si vuole realizzare una pipeline a tre stadi che elabora una sequenza di numeri interi. Gli stadi comunicano tra loro tramite canali `std::sync::mpsc`:

1. **Generatore** - produce una sequenza di `i64` (ad esempio i numeri da 1 a `N`) e li invia allo stadio successivo.
2. **Trasformatore** - per ciascun numero ricevuto, calcola il suo quadrato e lo inoltra allo stadio successivo.
3. **Collezionista** - riceve i numeri trasformati e li accumula in un `Vec<i64>` che restituisce al thread principale al termine dell'elaborazione.

Realizzare la funzione:

```
Rust
fn run_pipeline(n: i64) -> Vec<i64>
```

che esegue l'intera pipeline con `n` elementi prodotti e restituisce il vettore raccolto dal `Collector`.

Requisiti:

- Ciascuno dei tre stadi deve essere eseguito da un thread distinto.
- La terminazione della pipeline deve avvenire quando il **Generator** ha finito di produrre, il **Transformer** deve accorgersi della chiusura del canale di input e terminare a sua volta, e così via fino al **Collector**.
- L'ordine degli elementi nel vettore finale deve coincidere con quello di produzione.

Domande:

- Cosa cambia se si usa `mpsc::sync_channel(0)` al posto di `mpsc::channel()` tra **Generator** e **Transformer**? In che casi è preferibile?
- Come si potrebbe modificare la pipeline per propagare un eventuale errore dal **Generator** fino al thread principale?

Esercizio 4 - Cache concorrente con TTL

Si riprenda l'idea della cache TTL dell'esercizio 6 del Laboratorio 5 e la si estenda in modo che le voci scadute siano rimosse **automaticamente in background**, senza dover attendere il prossimo accesso a una chiave. La cache deve essere thread-safe e deve liberare correttamente le proprie risorse - incluso il thread di pulizia - al momento della distruzione.

Si definisca un tratto generico che modella l'interfaccia della cache, ed una struct che lo implementa. Una proposta di tratto è la seguente; firme leggermente diverse (per esempio sui tipi di ritorno o sui parametri presi per riferimento/valore) sono accettabili purché rispettino l'intento generale.

Rust

```
pub trait ConcurrentCache<V> {  
    fn new(ttl: Duration) -> Self where Self: Sized;  
    fn get(&self, key: &str) -> Option<Arc<V>>;  
    fn set(&self, key: &str, value: V);  
}  
  
pub struct ConcurrentCacheImpl<V> { /* campi */ }
```

Requisiti:

- Tutti i metodi devono essere thread-safe: più thread devono poter invocarli concorrentemente senza dare origine a corse critiche.
- Le voci scadute devono essere rimosse **automaticamente**, indipendentemente dal fatto che vengano poi richieste o meno tramite `get`.

- Quando l'istanza della cache viene distrutta (**drop**), il thread di pulizia in background deve terminare velocemente e in modo pulito.

Domande:

- Quale strategia avete scelto per lo strumento che sveglia il thread di pulizia ai momenti giusti? Quali altre alternative avreste avuto?
- Cosa accadrebbe se il thread di pulizia, ad ogni risveglio, mantenesse il lock di scrittura sulla mappa per tutta la durata della scansione delle voci? Come si potrebbe minimizzare il tempo di possesso del lock?
- Come gestireste invece una cache di tipo [LRU](#)? Quali modifiche dovrete applicare alla vostra struttura per cambiare il comportamento della cache attuale?

Esercizio 5 - Fan-out / fan-in con `crossbeam::channel`

Si vuole calcolare in parallelo, su una grande lista di numeri, una funzione costosa e poi raccogliere i risultati. L'architettura richiesta è la seguente:

- Un **dispatcher** (eseguito dal thread principale) inserisce i numeri di input in un canale MPMC.
- N thread **worker** consumano dal canale di input, calcolano la funzione costosa e producono il risultato in un secondo canale MPMC.
- Un thread **aggregator** consuma dal canale di output e accumula i risultati in un `Vec<i64, u32>` (coppia input/risultato).

Realizzare la funzione:

```
Rust
fn fan_out_fan_in(
    input: Vec<i64>,
    n_workers: usize
) -> Vec<i64, u32>
```

che restituisca un vettore contenente, per ciascun numero di `input`, una coppia (`numero`, `somma_delle_cifre_in_base_10`). L'ordine del vettore restituito non è specificato.

Requisiti:

- Si usino canali di `crossbeam_channel` con capacità a scelta, ma giustificata.
- Si usino esattamente `n_workers` thread worker.
- La terminazione deve essere naturale: la chiusura dei canali deve propagarsi in cascata dai worker all'aggregatore senza necessità di messaggi di terminazione.

Domande:

- Quali sono le differenze pratiche tra `std::sync::mpsc` e `crossbeam_channel`? In quali situazioni il secondo è preferibile?
- Che capacità conviene assegnare ai due canali? Cosa succederebbe con `bounded(0)`? E con `unbounded()`?
- Come ci si dovrebbe comportare se uno dei worker andasse in panico?